

OSGi Working Group
OSGi Specification repo for
org.osgi.service.enoceanRe-
lease 1.0.2-SNAPSHOT

September 2026



LICENSE

Eclipse Foundation Specification License – v1.0

By using and/or copying this document, or the Eclipse Foundation document from which this statement is linked, you (the licensee) agree that you have read, understood, and will comply with the following terms and conditions:

Permission to copy, and distribute the contents of this document, or the Eclipse Foundation document from which this statement is linked, in any medium for any purpose and without fee or royalty is hereby granted, provided that you include the following on ALL copies of the document, or portions thereof, that you use:

- link or URL to the original Eclipse Foundation document.
- All existing copyright notices, or if one does not exist, a notice (hypertext is preferred, but a textual representation is permitted) of the form: "Copyright © [date-of-document] Eclipse Foundation, Inc. <<url to this license>>"

Inclusion of the full text of this NOTICE must be provided. We request that authorship attribution be provided in any software, documents, or other items or products that you create pursuant to the implementation of the contents of this document, or any portion thereof.

No right to create modifications or derivatives of Eclipse Foundation documents is granted pursuant to this license, except anyone may prepare and distribute derivative works and portions of this document in software that implements the specification, in supporting materials accompanying such software, and in documentation of such software, PROVIDED that all such works include the notice below. HOWEVER, the publication of derivative works of this document for use as a technical specification is expressly prohibited.

The notice is:

"Copyright © [date-of-document] Eclipse Foundation. This software or document includes material copied from or derived from [title and URI of the Eclipse Foundation specification document]."

Disclaimers

THIS DOCUMENT IS PROVIDED "AS IS," AND THE COPYRIGHT HOLDERS AND THE ECLIPSE FOUNDATION MAKE NO REPRESENTATIONS OR WARRANTIES, EXPRESS OR IMPLIED, INCLUDING, BUT NOT LIMITED TO, WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE, NON-INFRINGEMENT, OR TITLE; THAT THE CONTENTS OF THE DOCUMENT ARE SUITABLE FOR ANY PURPOSE; NOR THAT THE IMPLEMENTATION OF SUCH CONTENTS WILL NOT INFRINGE ANY THIRD PARTY PATENTS, COPYRIGHTS, TRADEMARKS OR OTHER RIGHTS.

THE COPYRIGHT HOLDERS AND THE ECLIPSE FOUNDATION WILL NOT BE LIABLE FOR ANY DIRECT, INDIRECT, SPECIAL OR CONSEQUENTIAL DAMAGES ARISING OUT OF ANY USE OF THE DOCUMENT OR THE PERFORMANCE OR IMPLEMENTATION OF THE CONTENTS THEREOF.

The name and trademarks of the copyright holders or the Eclipse Foundation may NOT be used in advertising or publicity pertaining to this document or its contents without specific, written prior permission. Title to copyright in this document will at all times remain with copyright holders.

Preface

Implementation Requirements

An implementation of a Specification: (i) must fully implement the Specification including all its required interfaces and functionality; (ii) must not modify, subset, superset or otherwise extend the OSGi Name Space, or include any public or protected packages, classes, Java interfaces, fields or methods within the OSGi Name Space other than those required and authorized by the Specification. An implementation that does not satisfy limitations (i) (ii) is not considered an implementation of the Specification and must not be described as an implementation of the Specification. "OSGi Name Space" shall mean the public class or interface declarations whose names begin with "org.osgi" or any recognized successors or replacements thereof. An implementation of a Specification must not claim to be a compatible implementation of the Specification unless it passes the Technology Compatibility Kit ("TCK") for the Specification.

Feedback

This specification can be downloaded from the OSGi Documentation web site:

<https://docs.osgi.org/specification/>

Comments about this specification can be raised at:

<https://github.com/osgi/osgi/issues>

Table of Contents

1	Introduction	5
1.1	Reader Level	5
1.2	Version Information	5
1.3	References	5
139	Device Service Specification for EnOcean™ Technology	7
139.1	Introduction	7
139.2	Essentials	7
139.3	Entities	8
139.4	Operation Summary	9
139.5	EnOcean Base Driver	11
139.6	EnOcean Host	11
139.7	EnOcean Device	12
139.8	EnOcean Messages	13
139.9	EnOcean Message Description	14
139.10	EnOcean Channel	14
139.11	EnOcean Channel Description	15
139.12	EnOcean Remote Management	16
139.13	Working With an EnOcean Device	17
139.14	Event API	17
139.15	EnOcean Exceptions	18
139.16	Security	18
139.17	org.osgi.service.enocean	18
139.18	org.osgi.service.enocean.descriptions	25
139.19	References	29

Licensed under the Eclipse Foundation Specification License – v1.0. Copyright © Contributors to the Eclipse Foundation

DRAFT

1 Introduction

This document contains a specification developed by the OSGi working group.

1.1 Reader Level

This specification is written for the following audiences:

- Application developers
- Framework and system service developers (system developers)
- Architects

This specification assumes that the reader has at least one year of practical experience in writing Java programs. Experience with embedded systems and server-environments is a plus. Application developers must be aware that the OSGi environment is significantly more dynamic than traditional desktop or server environments.

System developers require a very deep understanding of Java. At least three years of Java coding experience in a system environment is recommended. A Framework implementation will use areas of Java that are not normally encountered in traditional applications. Detailed understanding is required of class loaders, garbage collection, Java 2 security, and Java native library loading.

Architects should focus on the introduction of each subject. This introduction contains a general overview of the subject, the requirements that influenced its design, and a short description of its operation as well as the entities that are used. The introductory sections require knowledge of Java concepts like classes and interfaces, but should not require coding experience.

Most of these specifications are equally applicable to application developers and system developers.

1.2 Version Information

Each OSGi specification has an independent version, and packages defined by those specifications have their own versions as well. Specification and package versions follow semantic versioning rules.

1.3 References

- [1] *OSGi Specifications*
<https://docs.osgi.org/specification/>

DRAFT

Licensed under the Eclipse Foundation Specification License – v1.0. Copyright © Contributors to the Eclipse Foundation

139 Device Service Specification for EnOcean™ Technology

Version 1.0

139.1 Introduction

EnOcean is a standard wireless communication protocol designed for low-cost and low-power devices by EnOcean Alliance.

EnOcean is widely supported by various types of devices such as smart meters, lights and many kinds of sensors in the residential area. OSGi applications need to communicate with those EnOcean devices. This specification defines how OSGi bundles can be developed to discover and control EnOcean devices on the one hand, and act as EnOcean devices and interoperate with EnOcean clients on the other hand. In particular, a Java mapping is provided for the standard representation of EnOcean devices called EnOcean Equipment Profile (EEPs). See [2] *EnOcean Equipment Profiles v2.6.2*.

The specification also describes the external API of an EnOcean Base Driver according to Device Access specification.

139.2 Essentials

- *Scope* - This specification is limited to general device discovery and control aspects of the standard EnOcean specifications. Aspects concerning the representation of specific or proprietary EnOcean profiles is not addressed.
- *Transparency* - EnOcean devices discovered on the network and devices locally implemented on the platform are represented in the OSGi service registry with the same API.
- *Lightweight implementation option* - The full description of EnOcean device services on the OSGi platform is optional. Some base driver implementations may implement all the classes including EnOcean device description classes while Implementations targeting constrained devices are able to implement only the part that is necessary for EnOcean device discovery and control.
- *Network Selection* - It must be possible to restrict the use of the EnOcean protocols to a selection of the connected devices.
- *Event handling* - Bundles are able to listen to EnOcean events.
- *Discover and control EnOcean devices as OSGi services* - Available learned (via an EnOcean teach-in procedure) EnOcean external endpoints are dynamically reified as OSGi services on the service registry upon discovery.
- *OSGi services as exported EnOcean devices* - OSGi services implementing the API defined here and explicitly set to be exported should be made available to networks with EnOcean-enabled endpoints in a transparent way.

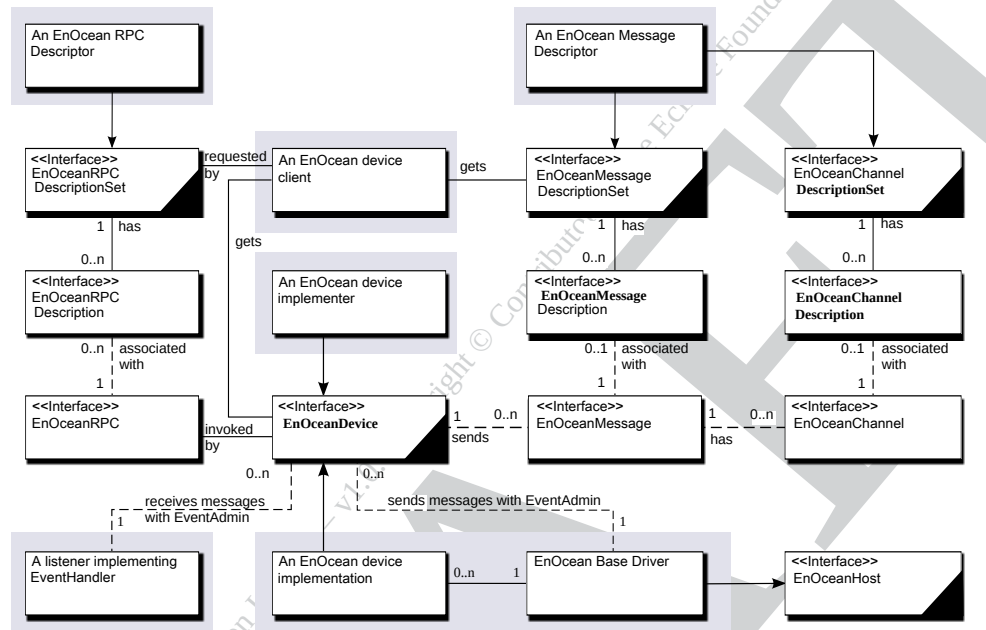
139.3 Entities

- *EnOcean Base Driver* - The bundle that implements the bridge between OSGi and EnOcean networks, see Figure 139.1 on page 9 . It is responsible for accessing the various EnOcean gateway chips on the execution machine, and ensures the reception and translation of EnOcean messages into proper objects. It is also used to send messages on the EnOcean network, using whatever chip it deems most appropriate.
- *EnOcean Host* - The `EnOceanHost` object is a link between the software and the EnOcean network. It represents the chip configuration (gateway capabilities) described in [5] *EnOcean System Specification - Security of EnOcean Radio Networks v1.9* . It is registered as an OSGi service.
- *EnOcean Device* - An EnOcean device. This entity is represented by a `EnOceanDevice` interface and registered as a service within the framework. It carries the unique chip ID of the device, and may represent either an imported or exported device, which may be a pure transmitter or a transceiver.
- *EnOcean Message* - Every EnOcean reporting equipment is supposed to follow a “profile”, which is essentially the way the emitted data is encoded. In order to reflect this standard as it is defined in [2] *EnOcean Equipment Profiles v2.6.2* , manufacturers are able to register the description of “Messages”, the essence of a profile, along with their associated payload (as Channels). See “EnOcean Channels” below for more information.
- *EnOcean Channel* - EnOcean channels are available as an array inside `EnOceanMessage` objects. They are a useful way to define any kind of payload that would be put inside of an EnOcean Message.

EnOcean Messages and their associated Channels can be described with `EnOceanMessageDescription` and `EnOceanChannelDescription` interfaces. Description providers aggregate these descriptions in sets that they register with `EnOceanMessageDescriptionSet` and `EnOceanChannelDescriptionSet` interfaces within the framework.

- *EnOcean RPC* - An interface that enables the invocation of vendor-specific Remote Procedure Calls and Remote Management Commands. These are particular types of Messages and are not linked to any EnOcean Profile, so that their descriptions are defined and registered in another way. The RPCs are documented via the `EnOceanRPCDescription` objects gathered into registered `EnOceanRPCDescriptionSet` services.
- *EnOcean Handler* - Enables clients to asynchronously get answers to their RPCs.
- *EnOcean Client* - An application that is intended to control EnOcean device services.
- *EnOcean Exception* - Delivers errors during `EnOceanMessage` serialization/deserialization or during execution outside transmission.

Figure 139.1 EnOcean Service Specification class diagram.

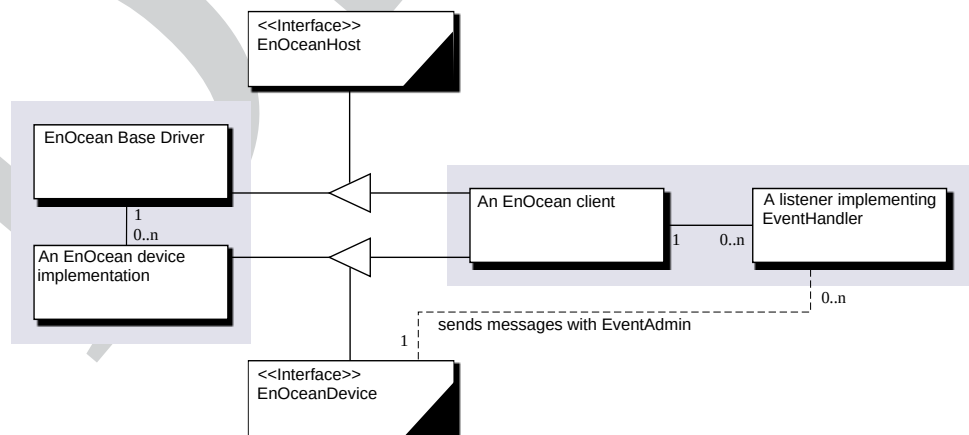


139.4 Operation Summary

To make an EnOcean device service available to EnOcean clients on the OSGi platform, it must be registered under the EnOceanDevice interface within the OSGi framework.

The EnOcean Base Driver is responsible for mapping external devices into EnOceanDevice objects, through the use of an EnOcean gateway. See [1] *Pervasive Service Composition in the Home Network*. The latter is represented on OSGi framework as an object implementing EnOceanHost interface. EnOcean “teach-in” messages will trigger this behavior, this is called a device import situation, see Figure 139.2 on page 9.

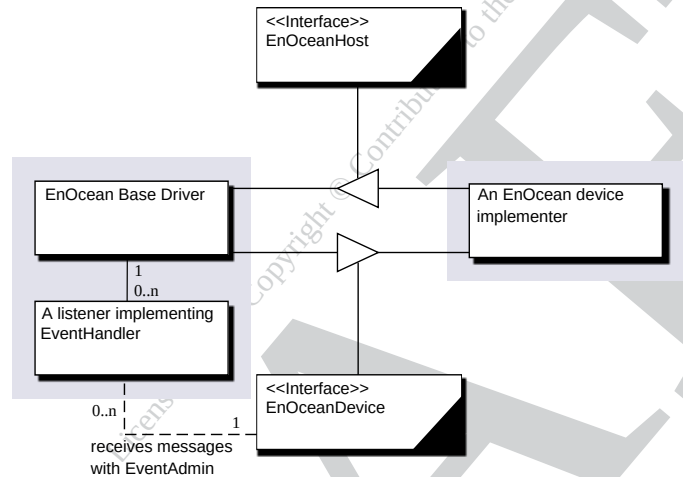
Figure 139.2 EnOcean device import.



Client bundles may also expose framework-internal (local) EnOceanDevice instances, registered within the framework, see Figure 139.3 on page 10. The Base Driver then should emulate those

objects as EnOcean devices on the EnOcean network. This is a device export situation, made possible by the use of the 127 virtual base IDs available on an EnOcean gateway. For more information about this process, see *Export Situation* on page 12 .

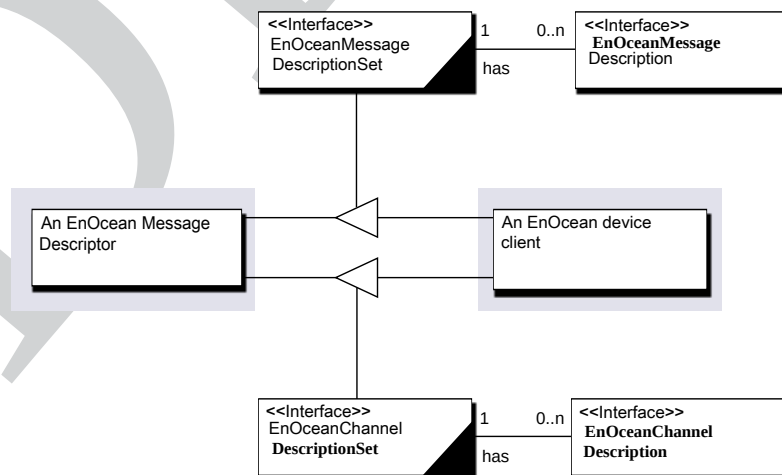
Figure 139.3 EnOcean device export.



EnOcean clients send RPCs (Remote Procedure Calls) to EnOcean devices and receives RPC responses and messages from them. Messages coming from EnOcean devices are accessible through Event Admin.

RPCs and messages content are specified by EnOcean Alliance or vendor-specific descriptions. Those descriptions may be provided on the OSGi platform by any bundle through the registration of `EnOceanRPCDescriptionSet`, `EnOceanMessageDescriptionSet` and `EnOceanChannelDescriptionSet` services. Every service is a set of description that enables applications to retrieve information about supported RPCs, messages or channels that compose messages.

Figure 139.4 Using a set of message descriptions.



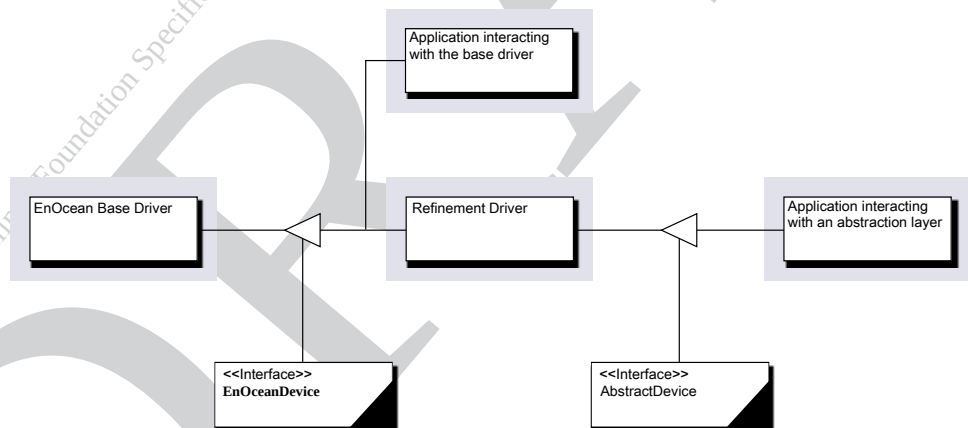
139.5 EnOcean Base Driver

Most of the functionality described in the operation summary is implemented in an EnOcean base driver. This bundle implements the EnOcean protocol and handles the interaction with bundles that use the EnOcean devices. An EnOcean base driver is able to discover EnOcean devices on the network and map each discovered device into an OSGi registered EnOceanDevice service. It is also the receptor, through EventAdmin service and OSGi service registry, of all the events related to local devices and clients. It enables bidirectional communication for RPC and Channel updates.

Several base drivers may be deployed on a residential OSGi device, one for every supported network technology. An OSGi device abstraction layer may then be implemented as a layer of refinement drivers above a layer of base drivers. The refinement driver is responsible for adapting technology-specific device services registered by the base driver into device services of another model, see AbstractDevice interface in Figure 139.5 on page 11. In the case of a generic device abstraction layer, the model is agnostic to technologies.

The EnOcean Alliance defines their own abstract model with EnOcean Equipment Profiles and refinement drivers may provide the implementation of all EEPs with EnOcean specific Java interfaces. The AbstractDevice interface of Figure 139.5 on page 11 is then replaced by an EEP specific Java interface in that case. The need and the choice of the abstraction depends on the targeted application domain.

Figure 139.5 *EnOcean Base Driver and a refinement driver representing devices in an abstract model.*



139.6 EnOcean Host

The EnOcean host represents an EnOcean gateway chip. Any EnOcean device service implementation should rely on at least one Gateway Chip in order to send and receive messages on the external EnOcean network. This interface enables standard control over an EnOcean compatible chip. Every EnOceanHost object should at least be identified by its unique chip ID.

The EnOceanHost interface enables OSGi applications to:

- Get or set gateway metadata (version, name, etc);
- Reset the gateway chip device;
- Retrieve a chip ID (derived from EnOcean's BASE_ID) for the given Service PID of a device.

139.7 EnOcean Device

139.7.1 Generics

A physical EnOcean device is reified as an [EnOceanDevice](#) object within the framework.

An EnOcean device holds most of the natural properties for an EnOcean object: its unique ID, the profile, a friendly name, its security information, and its available RPCs – along with the associated getters (and setters when applicable). All those properties MUST be persistent across restart so that teach-in procedures are made only once.

It also holds methods that reflect the natural actions a user application may physically trigger on such a device: send a message to the device, send a teach-in message to the device, or switch the device to learning mode.

Every EnOcean Device keeps a service PID property that is assigned either by the base driver or by any service-exporting bundle. The property value format is free and the value must be unique on the framework.

The properties on which EnOceanDevice services can be filtered on are: the device's service PID and chip ID, and its profile identifiers (RORG / FUNC / TYPE integers).

The EnOceanDevice also keeps security features as defined in the EnOcean Security Draft, [5] *EnOcean System Specification - Security of EnOcean Radio Networks v1.9*, which allow for a security level format (integer mask), an encryption key and/or a rolling authentication code.

The EnOceanDevice service MUST also be registered with the DEVICE_CATEGORY service property, see ??, that describes a array of categories to which the device belongs. One value MUST be EnOcean which is specified in [DEVICE_CATEGORY](#).

Values for the additional service properties, DEVICE_DESCRIPTION, DEVICE_SERIAL as defined in ??, are not specified here as no description nor application-level serial number are provided in the EnOcean standard protocol.

139.7.2 Import Situation

In import situations, the device's chip ID is uniquely set by the Base Driver, according to the one present in the teach-in message that originated the Device's creation. The service PID, see [7] *Persistent Identifier (PID)*, should also be generated and deterministically derived from the chip ID to allow reconstruction of a device without a new teach-in process after a framework restart.

139.7.3 Export Situation

In export situations:

1. The registering Client bundle sets the service PID of the EnOceanDevice object by itself, in a unique manner, and registers that object.
2. The chip ID (this device's EnOcean source ID when it issues messages) will be allocated by the Base Driver. The latter keeps a dictionary of the currently allocated chip IDs. The Client bundle must also set an [ENOCEAN_EXPORT](#) property in the registered device's Property Map.

The standard way to programmatically retrieve an exported chip ID from a given service PID is by using EnOceanHost's dedicated interface for this use.

The Base Driver MUST ensure the persistence of the CHIP_ID:SERVICE_PID mapping.

As an application developer, please refer to the documentation of your Base Driver to know its policies concerning exported chip ID updating, deletion and exhaustion.

139.7.4 Interface

The EnOceanDevice interface enables client bundles to:

- Get or set the security features of the device in a protected way;
- Retrieve the currently paired devices in the case of a receiver, as a collection of device IDs;
- Get the ID-based list of currently available RPCs for the device, as a Map of {manufacturerID:[functionId1, functionId2,...]};
- Invoke RPCs onto the device, through the `invoke(EnOceanRPC,EnOceanHandler)` call.

139.8 EnOcean Messages

EnOcean Messages are at the core of the EnOcean application layer as a whole and the EnOcean Equipment Profile specification, [2] *EnOcean Equipment Profiles v2.6.2*, in particular. Every exchange of information within EnOcean networks is done with a dedicated message. The [EnOceanMessage](#) interface provides a set of getters. The latter enables OSGi applications to get the information contained in the payload of an EnOcean message and defined as data and optional data of the EnOcean Serial Protocol Type 1 (RADIO) message (see Table 2 in Section "1.6.1 Packet description" of [4] *EnOcean System Specification - EnOcean Serial Protocol v1.17*).

This model enables reading both the EnOcean radio telegram data and the associated metadata that may be attached to it in a single object, `EnOceanMessage`.

In case the 'Optional Data' section gets missing at the lowest level (the radio access layer not following ESP protocol for instance) it is the responsibility of the Base Driver to mock the missing field's (dBm, destinationID, ...) values.

139.8.1 Mode of operation

Any `EnOceanMessage` object creation will be mirrored to Event Admin.

Details about the available topics, filters and properties can be found in *Event API* on page 17.

`EnOceanMessage` objects will be created only if the originating device has already been registered in the OSGi Service Registry, along with profile information.

139.8.2 Identification

The RORG of a message defines its shape and generic type; all the RORGs are defined in the EnOcean Radio Specification.

An addressed message will be encapsulated into an Addressed Telegram (ADT) by the base driver transparently; this means that from the application level, it will be represented under its original RORG, but with a valid destinationID.

A particular EnOcean Equipment Profile message is identified by three numbers: its RORG, and its FUNC, TYPE and EXTRA subtypes. In EnOcean, a (RORG, FUNC, TYPE) triplet is enough to identify a profile; though an EXTRA identifier is sometimes needed to identify a particular message layout for that profile.

Those identifiers allow for retrieving [EnOceanMessageDescription](#) objects within a registered [EnOceanMessageDescriptionSet](#), which give the application more information to parse the message.

139.8.3 Interface

The methods available in the [EnOceanMessage](#) interface are:

Service	Specification	for	EnOcean™	EnOcean Message Description Technology	Device Version	1.0
	• Identification methods, retrieving the message's profile, sender ID, optional destination ID, status; • A method to get the raw bytes of payload data in the message. This data can then be passed to the deserializer of the EnOceanMessageDescription object to be converted to EnOceanChannel, which may -again- be documented (through EnOceanChannelDescription objects) or not. • Link quality information read-only methods that mirror some of the 'Optional Data' header information.					

139.9 EnOcean Message Description

[EnOceanMessageDescription](#) objects exposes only two methods:

- [deserialize\(byte\[\]\)](#): makes the user able to deserialize the payload bytes of a raw [EnOceanMessage](#) object, into a collection of [EnOceanChannel](#) objects.
- [serialize\(EnOceanChannel\[\]\)](#): serializes the input [EnOceanChannel](#) objects into a collection of bytes.

139.10 EnOcean Channel

The [EnOceanChannel](#) interface is the first step of an abstraction to generate or interpret [EnOceanMessage](#) channels with plain Java types.

The simple [EnOceanChannel](#) interface provides a way to separate the different fields in a message payload, knowing their offset and size in the byte array that constitutes the full message's payload.

At the [EnOceanChannel](#) level, the only way to get/set the information contained in the channel is through a pair of [getRawValue\(\)](#) and [setRawValue\(byte\[\]\)](#) methods, which act on plain bytes.

Those bytes are meant right-aligned, and the number of those bytes is the size of the data field, floored up to the next multiple of 8. For instance, a 3-bit long channel would be encoded on one byte, all the necessary information starting from bit 0.

Every [EnOceanMessage](#) as described in the EEP Specification contains a various amount of channels, each of them being identified by their unique ID.

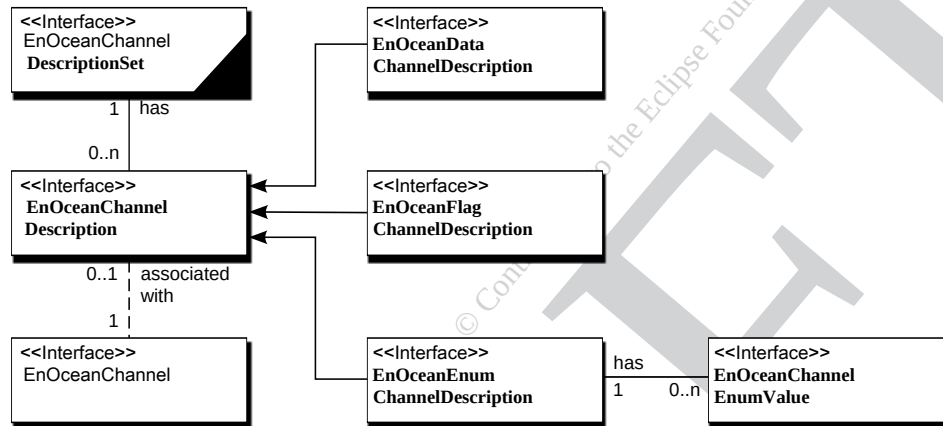
This ID, or channelID, is constituted of the "Shortcut" field of this channel from the EEP 2.5 Specification, [2] *EnOcean Equipment Profiles v2.6.2*, and a number fixed by the order of appearance of such a "Shortcut" in the specification.

This unique identifier links a Channel to an [EnOceanChannelDescription](#) object that provides more information to encode and decode that channel's information; see below for more details. This enables for loose coupling of the raw Channel itself and a richer, 3rd-party provided, information.

As an example, if the platform being developed is an electronic display that waits for Messages from a well-known temperature sensor, the Client bundle on the platform may interpret the Temperature Channels in every Temperature Message without needing an appropriate TemperatureChannelDescription object; it may directly cast and convert the `Byte[]` array of every received message to a properly valued `Double` and display that.

Otherwise, it could as well use the channelID to get a TemperatureChannelDescription object that would properly handle the deserialization process from the raw bytes to a proper, physical unit-augmented, result.

Figure 139.6 EnOcean channel and EnOcean channel descriptions.



139.11 EnOcean Channel Description

The [EnOceanChannelDescription](#) interface enables the description of all the various channels as specified in the EnOcean specification, as well as the description of channels issued by 3rd party actors.

Those description objects are retrieved from the registered [EnOceanChannelDescriptionSet](#) interface using an unique ID known as the channelID.

Here are the Channel types defined in this specification:

- **TYPE_RAW**: A collection of bytes. This type is used when the description is not provided, and is thus the default. For this type, the [deserialize\(byte\[\]\)](#) call actually returns a byte[] collection. The encryption key or a device ID on 4 bytes are examples of such raw types.
- **TYPE_DATA**: A scaled physical value. Used when the data can be mapped to a physical value; for instance, the 'WND – Wind Speed' channel is a raw binary value, in a range from 0 to 255, that will be mapped as a wind speed between 0 and 70 m/s. For this type, the [deserialize\(byte\[\]\)](#) call actually returns a Double value.
- **TYPE_FLAG**: A boolean value. Used when the Channel value can be either 1 or 0. The “Teach-In” Channel is a well-known example; this 1-bit field may either be 0 or 1, depending whether the Message is a teach-in one or not. For this type, the [deserialize\(byte\[\]\)](#) call actually returns a Boolean value.
- **TYPE_ENUM**: An enumeration of possible values. Used when the Channel can only take a discrete number of values. More complicated than **TYPE_FLAG**, enumerated types may have thresholds: for instance, the A5-30 “Digital Input- Input State (IPS)” channel is a 8-bit value which means “Contact closed” between 0 and 195, and “Contact open” from 196 to 255. For this type, the [deserialize\(byte\[\]\)](#) call actually returns an [EnOceanChannelEnumValue](#) object.

According to the channel type, the actual description object should implement one of the following specialized interfaces. This will ease the use of casting to the specialized interfaces on documented channels.

139.11.1 EnOcean Data Channel Description

The [EnOceanDataChannelDescription](#) interface inherits from [EnOceanChannelDescription](#) interface.

Two more methods give access to the integer input domain of the data channel (such as 0-255) and to the floating-point output range of it (such as -30.0°C – 24.5°C). A method is also present to retrieve the physical unit of the channel. The `serialize(Object)` and `deserialize(byte[])` methods are implemented to easily convert from the raw byte[] collection to a Double, and vice versa.

Here are a few samples of such Channels:

Table 139.1 EnOcean Data Channel Description example

Short	Description	Possible implemented name	Domain	Range	Unit
TMP	Temperature	TemperatureScaledChannel_X	0..255	-10°..+30°	°C
HUM	Humidity	HumidityScaledChannel_X	0..250	0..100	%

139.11.2 EnOcean Flag Channel Description

The `EnOceanFlagChannelDescription` interface inherits from the `EnOceanChannelDescription` interface.

Those channels, are typically used for On/Off reporting values (like a switch); they have no additional methods, though the `deserialize(byte[])` method converts the input bit into a proper Boolean object.

139.11.3 EnOcean Enumerated Channel Description

The `EnOceanEnumChannelDescription` interface inherits from the `EnOceanChannelDescription` interface.

The additional method provided to this interface is `getPossibleValues()`, which returns an array of the available `EnOceanChannelEnumValue` objects accessible to this channel. Every `EnOceanChannelEnumValue` object contains its integer input range and a String identifier that defines its meaning.

The `serialize(Object)` and `deserialize(byte[])` methods of an `EnOceanEnumChannelDescription` object thus convert an integer input value (say, 156) to an `EnOceanChannelEnumValue`, and vice versa.

Here is an example that shows the input range and the associated `EnOceanChannelEnumValue`:

Table 139.2 EnOcean Enumerated Channel Description example

Device profile	EnOceanChannelEnumValue	Start	Stop	Meaning
Fan speed stage switch	FanStageSwitch_Stage3	0	144	Fan speed: Stage 3
	FanStageSwitch_Stage2	145	164	Fan speed: Stage 2
	FanStageSwitch_Stage1	165	189	Fan speed: Stage 1
	FanStageSwitch_Stage0	190	209	Fan speed: Stage 0

139.12 EnOcean Remote Management

Remote Management is a feature which allows EnOcean devices to be configured and maintained over the air using radio messages.

The Remote Procedure Calls, or RPCs - as defined by the EnOcean Remote Management specification, [3] *EnOcean System Specification - Remote Management v2.0* - are not related to any EnOcean Equipment Profile.

Note that EnOcean Remote Commissioning is detailed in an additional EnOcean document, [6] *EnOcean Remote Commissioning Summary v1.0*.

139.12.1 EnOcean RPC

An [EnOceanRPC](#) object enables client bundles to remotely manage EnOcean devices using already defined behavior.

RPCs are defined by a [MANUFACTURER_ID](#) (11 bits, 0x7FF for the EnOcean alliance) and a unique [FUNCTION_ID](#) code on 12 bits.

RPCs are called directly onto an [EnOceanDevice](#) object via the [invoke\(EnOceanRPC, EnOceanHandler\)](#) method, which accepts also a non-mandatory [EnOceanHandler](#) object as a parameter to retrieve the asynchronous answer.

Broadcasted RPCs can be addressed directly to the Base Driver using the relevant Event Admin topic; see *Event API* on page 17 .

139.12.2 EnOcean Handler

Responses to RPCs are processed by the driver and sent back to a handler using [notifyResponse\(EnOceanRPC, byte\[\]\)](#) method when an [EnOceanHandler](#) is passed to the base driver.

139.13 Working With an EnOcean Device**139.13.1 Service Tracking**

All discovered EnOcean devices in the local networks are registered under [EnOceanDevice](#) interface within the OSGi framework. Every time an EnOcean device appears or quits the network, the associated OSGi service is registered or unregistered in the OSGi service registry. Thanks to the EnOcean Base Driver, the OSGi service availability in the registry mirrors EnOcean device availability on EnOcean network, [1] *Pervasive Service Composition in the Home Network* .

Thanks to service events, a bundle is able to track the addition, modification and removal of an [EnOceanDevice](#) service.

The following example shows using a ServiceTracker to track EnOceanDevice services.

```
ServiceTracker<EnOceanDevice, EnOceanDevice> enOceanTracker =
    new ServiceTracker<>(bundleContext, EnOceanDevice.class, null);
enOceanTracker.open(); // open the tracker
```

```
...
```

```
// get a snapshot of the current EnOceanDevice services
EnOceanDevice[] enOceanDeviceSnapshot =
    enOceanTracker.getServices(new EnOceanDevice[0]);
```

```
...
```

```
enOceanTracker.close(); // close the tracker
```

139.14 Event API

EnOcean events must be delivered to the EventAdmin service by the EnOcean implementation, if present. EnOcean event topic follow the following form: `org/osgi/service/enOcean/EnOceanEvent/SUBTOPIC`.

MESSAGE_RECEIVED and RPC_BROADCAST are the two available subtopics.

139.14.1**MESSAGE_RECEIVED**

Properties (every event may dispatch some or all of the following properties):

- [CHIP_ID](#) – The chip ID of the sending device.
- [service.pid](#) – The service PID of the exported device.
- [RORG](#) – The RORG (Radio Telegram Type) of the sending device.
- [FUNC](#) – The FUNC profile identifier of the sending device.
- [TYPE](#) – The TYPE profile identifier of the sending device.
- [PROPERTY_MESSAGE](#) – The EnOceanMessage object associated with this event.
- [PROPERTY_EXPORTED](#) – The presence of this property means that this message has actually been exported from a locally implemented EnOcean Device.

139.14.2**RPC_BROADCAST**

This event is used whenever an RPC is broadcasted on EnOcean networks, in IMPORT or EXPORT situations.

Properties (every event may dispatch some or all of the following properties):

- [MANUFACTURER_ID](#) – The RPC's manufacturer ID.
- [FUNCTION_ID](#) – The RPC's function ID.
- [PROPERTY_EXPORTED](#) – The presence of this property means that this RPC has actually been exported from a locally implemented EnOcean Device.
- [PROPERTY_RPC](#) – The EnOceanRPC object associated with this event.

139.15**EnOcean Exceptions**

The [EnOceanException](#) can be thrown and holds information about the different EnOcean layers. Here below, ESP stands for *EnOcean Serial Protocol*. The following errors are defined:

- [ESP_UNEXPECTED_FAILURE](#) – Operation was not successful.
- [ESP_RET_NOT_SUPPORTED](#) – The ESP command was not supported by the driver.
- [ESP_RET_WRONG_PARAM](#) – The ESP command was supplied wrong parameters.
- [ESP_RET_OPERATION_DENIED](#) – The ESP command was denied authorization.
- [INVALID_TELEGRAM](#) – The message was invalid.

139.16**Security**

It is recommended that `ServicePermission[EnOceanDevice|EnOceanHost, REGISTER|GET]` be used sparingly and only for bundles that are trusted.

139.17**org.osgi.service.enocean**

EnOcean Package Version 1.0.

Bundles wishing to use this package must list the package in the Import-Package header of the bundle's manifest. This package has two types of users: the consumers that use the API in this package and the providers that implement the API in this package.

Example import for consumers using the API in this package:

```
{@code Import-Package: org.osgi.service.enocean; version="[1.0,2.0)"}
```

Example import for providers implementing the API in this package:

```
{@code Import-Package: org.osgi.service.enocean; version="[1.0,1.1)"}
```

139.17.1 Summary

- `EnOceanChannel` - Holds the raw value and channel identification info of an `EnOceanChannel`.
- `EnOceanDevice` - This interface represents a physical device that communicates over the EnOcean protocol.
- `EnOceanEvent` - Constants for use in `EnOcean` events.
- `EnOceanException` - This class contains code and definitions necessary to support common `EnOcean` exceptions.
- `EnOceanHandler` - The interface used to get callback answers from a RPC or a Message.
- `EnOceanHost` - This interface represents an `EnOcean` Host, a device that offers `EnOcean` networking features.
- `EnOceanMessage` - Holds the necessary methods to interact with an `EnOcean` message.
- `EnOceanRPC` - A very basic interface for RPCs.

139.17.2 **public abstract EnOceanChannel**

Holds the raw value and channel identification info of an `EnOceanChannel`.

139.17.2.1 **public abstract String getChannelId()**

Returns The unique ID of this channel.

139.17.2.2 **public abstract int getOffset()**

Returns The offset, in bits, where this channel is found in the telegram.

139.17.2.3 **public abstract byte[][] getRawValue()**

- Gets the raw value of this channel.

Returns corresponding value.

139.17.2.4 **public abstract int getSize()**

Returns The size, in bits, of this channel.

139.17.2.5 **public abstract void setRawValue(byte[][] rawValue)**

rawValue

- Sets the raw value of a channel.

139.17.3 **public abstract EnOceanDevice**

This interface represents a physical device that communicates over the `EnOcean` protocol.

139.17.3.1 **public static final String CHIP_ID = "enocean.device.chip_id"**

Property name for the mandatory `CHIP_ID` of the device

139.17.3.2 **public static final String DEVICE_CATEGORY = "EnOcean"**

Property name for the mandatory `DEVICE_CATEGORY` of the device

139.17.3.3 **public static final String ENOCEAN_EXPORT = "enocean.device.export"**

Property name that defines if the device is exported or not. If present, the device is exported.

- 139.17.3.4** **public static final String FUNC = "enocean.device.profile.func"**
Property name for the radiotelegram functional type of the profile associated with this device.
- 139.17.3.5** **public static final String MANUFACTURER = "enocean.device.manufacturer"**
Property name for the manufacturer ID that may be specified by some teach-in messages.
- 139.17.3.6** **public static final String RORG = "enocean.device.profile.rorg"**
Property name for the radiotelegram main type of the profile associated with this device.
- 139.17.3.7** **public static final String SECURITY_LEVEL_FORMAT = "enocean.device.security_level_format"**
Property name for the security level mask for this device. The format of that mask is specified in EnOcean Security Draft.
- 139.17.3.8** **public static final String TYPE = "enocean.device.profile.type"**
Property name for the radiotelegram subtype of the profile associated with this device.
- 139.17.3.9** **public abstract int getChipId()**
Returns The EnOcean device chip ID.
- 139.17.3.10** **public abstract byte[][] getEncryptionKey()**
□ Returns the current encryption key used by this device.
Returns The current encryption key, or null.
- 139.17.3.11** **public abstract int getFunc()**
Returns The EnOcean profile FUNC, or -1 if unknown.
- 139.17.3.12** **public abstract int[][] getLearnedDevices()**
□ Gets the list of devices the device already has learned.
Returns The list of currently learned device's CHIP_IDs.
- 139.17.3.13** **public abstract int getManufacturer()**
Returns The EnOcean manufacturer code, -1 if unknown.
- 139.17.3.14** **public abstract int getRollingCode()**
□ Get the current rolling code of the device.
Returns The current rolling code in use with this device's communications.
- 139.17.3.15** **public abstract int getRorg()**
Returns The EnOcean profile RORG.
- 139.17.3.16** **public abstract Map<Integer, Integer[]> getRPCs()**
□ Retrieves the currently available RPCs to this device; those are stored using their manufacturerId:commandId identifiers.
Returns A list of the available RPCs, in a {@code Map<Integer, Integer[]>} form.
- 139.17.3.17** **public abstract int getSecurityLevelFormat()**
Returns The EnOcean security level format, or 0 as default (no security)
- 139.17.3.18** **public abstract int getType()**
Returns The EnOcean profile TYPE, or -1 if unknown.

139.17.3.19 public abstract void invoke(EnOceanRPC rpc, EnOceanHandler handler)*rpc**handler*

- Sends an RPC to the remote device.

Throws IllegalArgumentException–**139.17.3.20 public abstract void remove()**

- Removes the device's OSGi service from OSGi service platform.

139.17.3.21 public abstract void setEncryptionKey(byte[][] key)*key* the encryption key to be set.

- Sets the encryption key of the device.

139.17.3.22 public abstract void setFunc(int func)*func* the EEP func of the device;

- Manually sets the EEP FUNC of the device.

139.17.3.23 public abstract void setLearningMode(boolean learnMode)*learnMode* the desired state: true for learning mode, false to disable it.

- Switches the device into learning mode.

139.17.3.24 public abstract void setRollingCode(int rollingCode)*rollingCode* the rolling code to be set or initiated.

- Sets the rolling code of this device.

139.17.3.25 public abstract void setType(int type)*type* the EEP type of the device;

- Manually sets the EEP TYPE of the device.

139.17.4 public final class EnOceanEvent

Constants for use in EnOcean events.

139.17.4.1 public static final String PROPERTY_EXPORTED = "enocean.message.is_exported"

Property key used to tell apart messages that are exported or imported.

139.17.4.2 public static final String PROPERTY_MESSAGE = "enocean.message"

Property key for the {@link EnOceanMessage} object embedded in an event.

139.17.4.3 public static final String PROPERTY_RPC = "enocean.rpc"

Property key for the {@link EnOceanRPC} object embedded in an event.

139.17.4.4 public static final String TOPIC_MSG_RECEIVED = "org/osgi/service/enocean/EnOceanEvent/MESSAGE_RECEIVED"

Main topic for all OSGi dispatched EnOcean messages, imported or exported.

139.17.4.5 public static final String TOPIC_RPC_BROADCAST = "org/osgi/service/enocean/EnOceanEvent/RPC_BROADCAST"

Main topic for all OSGi broadcast EnOcean RPCs, imported or exported.

139.17.5 **public class EnOceanException** **extends Exception**

This class contains code and definitions necessary to support common EnOcean exceptions. This class is mostly used with low-level, gateway-interacting code : EnOceanHost.

139.17.5.1 **public static final short ESP_RET_NOT_SUPPORTED = 2**

Operation is not supported by the target device.

139.17.5.2 **public static final short ESP_RET_OPERATION_DENIED = 4**

The operation was denied.

139.17.5.3 **public static final short ESP_RET_WRONG_PARAM = 3**

One of the parameters was badly specified or missing.

139.17.5.4 **public static final short ESP_UNEXPECTED_FAILURE = 1**

Unexpected failure.

139.17.5.5 **public static final short INVALID_TELEGRAM = 240**

The message was invalid.

139.17.5.6 **public static final short SUCCESS = 0**

SUCCESS status code.

139.17.5.7 **public EnOceanException(String errorDesc)**

errorDesc exception error description

- Constructor for EnOceanException

139.17.5.8 **public EnOceanException(int errorCode, String errorDesc)**

errorCode the error code.

errorDesc the description.

- Constructor for EnOceanException

139.17.5.9 **public EnOceanException(int errorCode)**

errorCode An error code.

- Constructor for EnOceanException

139.17.5.10 **public int errorCode()**

- Constructor for EnOceanException

Returns An EnOcean error code, defined by the EnOcean Forum working committee or an EnOcean vendor.

139.17.6 **public abstract EnOceanHandler**

The interface used to get callback answers from a RPC or a Message.

139.17.6.1 **public abstract void notifyResponse(EnOceanRPC original, byte[][] payload)**

original the original {@link EnOceanRPC} that originated this answer.

payload the payload of the response; may be deserialized to an {@link EnOceanRPC} object.

- Notifies of the answer to a RPC.

139.17.7 public abstract EnOceanHost

This interface represents an EnOcean Host, a device that offers EnOcean networking features.

139.17.7.1 public static final Object HOST_ID

The unique ID for this Host: this matches the CHIP_ID of the EnOcean Gateway Chip it embodies.

139.17.7.2 public static final int REPEATER_LEVEL_OFF = 0

repeater level to disable repeating; this is the default.

139.17.7.3 public static final int REPEATER_LEVEL_ONE = 1

repeater level to repeat every telegram at most once.

139.17.7.4 public static final int REPEATER_LEVEL_TWO = 2

repeater level to repeat every telegram at most twice.

139.17.7.5 public abstract String apiVersion() throws EnOceanException

- Returns the chip's API version info (cf. ESP3 command 0x03: CO_RD_VERSION)

Returns a String object containing the API version info.

Throws EnOceanException – if any problem occurs.

139.17.7.6 public abstract String appVersion() throws EnOceanException

- Returns the chip's application version info (cf. ESP3 command 0x03: CO_RD_VERSION)

Returns a String object containing the application version info.

Throws EnOceanException – if any problem occurs.

139.17.7.7 public abstract int getBaseID() throws EnOceanException

- Gets the BASE_ID of the chip, if set (cf. ESP3 command 0x08: CO_RD_IDBASE)

Returns the BASE_ID of the device as defined in EnOcean specification

Throws EnOceanException – if any problem occurs.

139.17.7.8 public abstract int getChipId(String servicePID) throws EnOceanException

servicePID

- Retrieves the CHIP_ID associated with the given servicePID, if existing on this chip.

Returns the associated CHIP_ID of the exported device.

Throws EnOceanException – if any problem occurs.

139.17.7.9 public abstract int getRepeaterLevel() throws EnOceanException

- Gets the current repeater level of the host (cf. ESP3 command 0x0A: CO_RD_REPEATER)

Returns one of the Repeater Level constants as defined above.

Throws EnOceanException – if any problem occurs.

139.17.7.10 public abstract void reset() throws EnOceanException

- Reset the EnOcean Host (cf. ESP3 command 0x02: CO_WR_RESET)

Throws EnOceanException – if any problem occurs.

139.17.7.11 public abstract void setBaseID(int baseID) throws EnOceanException

baseID to be set.

- Sets the base ID of the device, may be used up to 10 times (cf. ESP3 command 0x07: CO_WR_ID-BASE)

Throws EnOceanException – if any problem occurs.

139.17.7.12 **public abstract void setRepeaterLevel(int level) throws EnOceanException**

level one of the Repeater Level constants as defined above.

- Sets the repeater level on the host (cf. ESP3 command 0x09: CO_WR_REPEATER)

Throws EnOceanException – if any problem occurs.

139.17.8 **public abstract EnOceanMessage**

Holds the necessary methods to interact with an EnOcean message.

139.17.8.1 **public abstract byte[][] getBytes()**

- Gets the bytes corresponding to the whole message, including the CRC. The generated byte[] array may be sent to an EnOcean gateway and is conform to EnOcean Radio Protocol.

Returns The serialized byte list corresponding to the binary message.

139.17.8.2 **public abstract int getDbm()**

- Returns the average RSSI on all the received subtelegrams, including redundant ones.

Returns The average RSSI perceived.

139.17.8.3 **public abstract int getDestinationId()**

Returns the message's destination ID, or -1

139.17.8.4 **public abstract int getFunc()**

Returns the message's FUNC

139.17.8.5 **public abstract byte[][] getPayloadBytes()**

- Returns the payload bytes of this message.

Returns corresponding value.

139.17.8.6 **public abstract int getRorg()**

Returns the message's RORG

139.17.8.7 **public abstract int getSecurityLevelFormat()**

- Returns the security level of this message, as specified in the 'Security of EnOcean Radio Networks' draft, section 4.2.1.3.

Returns The security level format.

139.17.8.8 **public abstract int getSenderId()**

Returns the message's Sender ID

139.17.8.9 **public abstract int getStatus()**

- Gets the current EnOcean status of the Message. The 'status' byte is actually a bitfield that mainly holds repeater information, teach-in status, and more or less information depending on the radiotelegram type.

Returns the current EnOcean status of this message.

139.17.8.10 **public abstract int getSubTelNum()**

- Returns the number of subtelegrams (usually 1) this Message carries.

Returns The number of subtelegrams in the case of multiframe messages.

139.17.8.11 **public abstract int getType()**

Returns the message's TYPE

139.17.9 **public abstract EnOceanRPC**

A very basic interface for RPCs.

139.17.9.1 **public static final String FUNCTION_ID = "enocean.rpc.function_id"**

The Function ID property string, used in EventAdmin RPC broadcasting.

139.17.9.2 **public static final String MANUFACTURER_ID = "enocean.rpc.manufacturer_id"**

The Manufacturer ID property string, used in EventAdmin RPC broadcasting.

139.17.9.3 **public abstract int getFunctionId()**

- Gets the functionID for this RPC.

Returns function id.

139.17.9.4 **public abstract int getManufacturerId()**

- Gets the manufacturerID for this RPC.

Returns manufacturer id.

139.17.9.5 **public abstract String getName()**

- Get a friendly name for the RPC

Returns the name.

139.17.9.6 **public abstract byte[][] getPayload()**

- Gets the current payload of the RPC.

Returns the payload, in bytes, of this RPC.

139.17.9.7 **public abstract int getSenderId()**

- Sets the RPC's senderID. This member has to belong to {@link EnOceanRPC} interface, for the object may be sent as a standalone using EventAdmin for instance.

Returns sender id.

139.17.9.8 **public abstract void setSenderId(int chipId)**

chipId

- Sets the RPC's senderID.

139.18 **org.osgi.service.enocean.descriptions**

EnOcean Descriptions Package Version 1.0.

Bundles wishing to use this package must list the package in the Import-Package header of the bundle's manifest. This package has two types of users: the consumers that use the API in this package and the providers that implement the API in this package.

139.18.1

Summary

- `EnOceanChannelDescription` - Public and registered description interface for a channel.
- `EnOceanChannelDescriptionSet` - This interface represents an EnOcean Channel Description Set.
- `EnOceanChannelEnumValue` - This transitional interface is used to define all the possible values taken by an enumerated channel.
- `EnOceanDataChannelDescription` - Subinterface of {@link `EnOceanChannelDescription`} that describes physical measuring channels.
- `EnOceanEnumChannelDescription` - Subinterface of {@link `EnOceanChannelDescription`} that describes enumerated channels.
- `EnOceanFlagChannelDescription` - Subinterface of {@link `EnOceanChannelDescription`} that describes boolean channels.
- `EnOceanMessageDescription` - This interface represents an EnOcean Message Description.
- `EnOceanMessageDescriptionSet` - This interface represents an EnOcean Message Description Set.

139.18.2

public abstract `EnOceanChannelDescription`

Public and registered description interface for a channel. Encompasses all the possible subtypes for a channel.

139.18.2.1

public static final String CHANNEL_ID = "enocean.channel.description.channel_id"

The unique ID of this `EnOceanChannelDescription` object.

139.18.2.2

public static final String TYPE_DATA = "enocean.channel.description.data"

A DATA channel maps itself to a {@link `Double`} value representing a physical measure.

139.18.2.3

public static final String TYPE_ENUM = "enocean.channel.description.enum"

An ENUM channel maps itself to one between a list of discrete {@link `EnOceanChannelEnumValue`} "value objects".

139.18.2.4

public static final String TYPE_FLAG = "enocean.channel.description.flag"

A FLAG channel maps itself to a {@link `Boolean`} value.

139.18.2.5

public static final String TYPE_RAW = "enocean.channel.description.raw"

A RAW channel is only made of bytes.

139.18.2.6

public abstract Object deserialize(byte[][] bytes)

bytes the right-aligned raw bytes.

- Tries to deserialize a series of bytes into a documented value object (raw bytes, `Double` or {@link `EnOceanChannelEnumValue`}. Of course this method will be specialized for each {@link `EnOceanChannelDescription`} subinterface, depending on the type of this channel.

Returns a value object.

Throws `IllegalArgumentException`–

139.18.2.7 public abstract String getType()

- Retrieves the type of the channel.

Returns one of the above-described types.

139.18.2.8 public abstract byte[][] serialize(Object obj)

obj the value of the channel.

- Tries to serialize the channel into a series of bytes.

Returns the right-aligned value, in raw bytes, of the channel.

Throws IllegalArgumentException–

139.18.3 public abstract EnOceanChannelDescriptionSet

This interface represents an EnOcean Channel Description Set. {@link EnOceanChannelDescriptionSet} is registered as an OSGi Service. Provides a method to retrieve the {@link EnOceanChannelDescription} objects it documents.

139.18.3.1 public abstract EnOceanChannelDescription getChannelDescription(String channelId)

channelId the unique string identifier of the description object.

- Retrieves a {@link EnOceanChannelDescription} object according to its identifier.

Returns The corresponding {@link EnOceanChannelDescription} object, or null.

Throws IllegalArgumentException– if the supplied String is invalid, null, or other reason.

139.18.4 public abstract EnOceanChannelEnumValue

This transitional interface is used to define all the possible values taken by an enumerated channel.

139.18.4.1 public abstract String getDescription()

- A non-mandatory description of what this enumerated value is about.

Returns the description of this channel.

139.18.4.2 public abstract int getStart()

- The start value of the enumeration.

Returns the start value.

139.18.4.3 public abstract int getStop()

- The stop value of the enumeration.

Returns the stop value.

**139.18.5 public abstract EnOceanDataChannelDescription
extends EnOceanChannelDescription**

Subinterface of {@link EnOceanChannelDescription} that describes physical measuring channels.

139.18.5.1 public abstract int getDomainStart()

- The start of the raw input range for this channel.

Returns the domain start.

139.18.5.2 public abstract int getDomainStop()

- The end of the raw input range for this channel.

Service	Specification	for	EnOcean™	org.osgi.service.enocean.descriptions Technology	Device Version	1.0
	<i>Returns</i>	the domain stop.				
139.18.5.3	public abstract double getRangeStart()					
	□ The scale start at which this channel will be mapped to (-20,0°C for instance)					
	<i>Returns</i>	the range start.				
139.18.5.4	public abstract double getRangeStop()					
	□ The scale stop at which this channel will be mapped to (+30,0°C for instance)					
	<i>Returns</i>	the range stop.				
139.18.5.5	public abstract String getUnit()					
	□ The non-mandatory physical unit description of this channel.					
	<i>Returns</i>	the unit as a String				
139.18.6	public abstract EnOceanEnumChannelDescription extends EnOceanChannelDescription					
	Subinterface of {@link EnOceanChannelDescription} that describes enumerated channels.					
139.18.6.1	public abstract EnOceanChannelEnumValue[][] getPossibleValues()					
	□ Gets all the possible value for this channel.					
	<i>Returns</i>	corresponding value(s).				
139.18.7	public abstract EnOceanFlagChannelDescription extends EnOceanChannelDescription					
	Subinterface of {@link EnOceanChannelDescription} that describes boolean channels.					
139.18.8	public abstract EnOceanMessageDescription					
	This interface represents an EnOcean Message Description.					
139.18.8.1	public abstract EnOceanChannel[][] deserialize(byte[][] bytes)					
	<i>bytes</i>	to be deserialized.				
	□ Deserializes an array of bytes into the EnOceanChannels available to the payload, if possible.					
	<i>Returns</i>	deserialized value.				
	<i>Throws</i>	IllegalArgumentException— if the actual instance type of the message is not compatible with the bytes it is fed with (RORG to begin with).				
139.18.8.2	public abstract String getMessageDescription()					
	<i>Returns</i>	the message description containing the RORG, (and the FUNC, and the TYPE if available), as well as, the EEP's "title" (e.g. for F60201: Rocker Switch, 2 Rocker; Light and Blind Control - Application Style 1).				
139.18.8.3	public abstract byte[][] serialize(EnOceanChannel[][] channels)					
	<i>channels</i>	to be serialized.				
	□ Serializes a series of {@link EnOceanChannel} objects into the corresponding byte[] sequence.					
	<i>Returns</i>	serialized value.				
	<i>Throws</i>	IllegalArgumentException— if the given channels is null.				

139.18.9 public abstract EnOceanMessageDescriptionSet

This interface represents an EnOcean Message Description Set. {@link EnOceanMessageDescriptionSet} is registered as an OSGi Service. Provides method to retrieve the {@link EnOceanMessageDescription} objects it documents.

139.18.9.1 public abstract EnOceanMessageDescription getMessageDescription(int rorg, int func, int type, int extra)

rorg the radio telegram type of the message.

func The func subtype of this message.

type The type subselector.

extra Some extra information; some {@link EnOceanMessageDescription} objects need an additional specifier. If not needed, has to be set to -1.

- Retrieves a {@link EnOceanMessageDescription} object according to its identifiers. See EnOcean Equipment Profile Specification for more details.

Returns The {@link EnOceanMessageDescription} object looked for, or null.

Throws IllegalArgumentException – if there was an error related to the input arguments.

139.19 References

- [1] *Pervasive Service Composition in the Home Network*
Bottaro, A., Gérodolle, A., Lalanda, P., 21st IEEE International Conference on Advanced Information Networking and Applications (AINA-07), Niagara Falls, Canada, May 2007
- [2] *EnOcean Equipment Profiles v2.6.2*
EnOcean Alliance, https://www.enocean-alliance.org/en/enocean_standard/ , November 19, 2014
- [3] *EnOcean System Specification - Remote Management v2.0*
EnOcean Alliance, March 06, 2014
- [4] *EnOcean System Specification - EnOcean Serial Protocol v1.17*
EnOcean Alliance, August 2, 2011
- [5] *EnOcean System Specification - Security of EnOcean Radio Networks v1.9*
EnOcean Alliance, July 26, 2013
- [6] *EnOcean Remote Commissioning Summary v1.0*
EnOcean Alliance, <https://www.enocean-alliance.org/en/downloads/> , December 01, 2014
- [7] *Persistent Identifier (PID)*
OSGi Core Release, Service Layer

DRAFT

Licensed under the Eclipse Foundation Specification License – v1.0. Copyright © Contributors to the Eclipse Foundation

End Of Document

Licensed under the Eclipse Foundation Specification License – v1.0. Copyright © Contributors to the Eclipse Foundation

DRAFT